

Claude Chat on the web: first product surface

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 1 of 12 · English notes

After the theory block, Vijay opened the **Claude desktop / Chat** surface—the product he analogised to a ChatGPT-style conversational assistant. On desktop home he showed two tiles: **Chat** and **Cowork**. This chapter stays on Chat: how a web or desktop chat client talks to a stateless model, which controls matter for tokens, and how files, projects, skills, and connectors change the request.

What Chat is for (and what it will not do)

Chat is the everyday conversational client: questions, writing, analysis, uploaded images, and (with tools) web search. A later Q&A made the product boundary sharp: **Chat can answer in the same window; it does not reorganise files on your disk**. That job is Cowork. If you only need a summary of a slide deck you uploaded, Chat is enough. If you need month-wise folders written next to the originals, you have left Chat.

Surface	Where you typically open it	Local file create / move / delete	Typical user
Claude Chat	claude.ai web, desktop Chat pane	No — replies stay in the thread (unless you download an artefact)	Anyone: BA, sales, developer exploring an idea
Claude Cowork	Desktop (org + version dependent)	Yes — on an attached folder, with approval modes	Knowledge work and local multi-step tasks
Claude Code	CLI, VS Code / other IDEs, desktop Code	Yes — in the repo / workspace via tools	Developers on CCDV-F

YOUR ORG CHAT MAY NOT LOOK LIKE THE DEMO

On 5 August several UST attendees had Chat and Code on desktop but no Cowork: either an old desktop build or an org policy (IT later said some R&D / VDI workspaces enable Cowork). Chat itself can still be gated. Trust what your admin enabled and what docs.claude.com describes today.

Model picker, effort, and extended thinking

In Chat you choose a model (Haiku / Sonnet / Opus family in class). Vijay deferred “when to pick which” to later sessions, but he already tied **effort** to token budget: low / medium / high / extra / max, with max described as opening a ~1 million token path. **Thinking** (deep / extended thinking) routes work to more expensive reasoning SKUs. Do not enable max effort and thinking “because it looks smarter” on a one-word lookup; you will burn quota that your team shares.

He demonstrated asking the model its training cutoff. Chat answered with a date and then offered to look things up—the natural cue to turn on **web search** as a tool. For a live “weather in Bengaluru” style prompt, the model decides a tool is needed; the *client* performs the HTTP; JSON returns; a second model call writes

prose. Planning tokens plus synthesis tokens both count. Pure HTTP to a weather API, once planned, is not billed as an LLM call.

Files versus Projects (RAG without saying RAG)

Attaching a file or photo to a Chat turn typically **appends that content** into the current request (subject to window limits). Creating a **Project** and dropping many files is different: Vijay described it as the product quietly treating the workspace as **RAG**—index, retrieve chunks, augment the prompt, generate. You do not pick the chunker or the vector store; that is proprietary to Claude. Cursor-class tools may expose those knobs; Claude Chat/Code in this course do not.

- Educate the tool in `CLAUDE.md` / project instructions: e.g. “front end follows this OpenAPI spec” so retrieval does not haul the entire backend.
- If Swagger lives in another repo, you may need the URL or the JSON file in context—not a live HTTP connector unless you add one.
- Long chats accumulate prior Q&A into every new turn. Start a **new session** when the task is done to limit tokens and hallucination hooks. Some clients offer **summarize / compact** so later turns send history + new question instead of the full transcript.

Skills, slash commands, and connectors in Chat

To fight the “where is the bank?” ambiguity, Chat can inject **skills** (reusable system instructions). The user prompt stays short (“where is the bank?”); the skill may be a thousand lines of banking policy. Those lines are tokens every time. Slash commands (e.g. a geolocation skill versus a banking skill) swap which instruction pack is attached. Connectors (Gmail, Drive, Jira, Confluence, Microsoft 365, Slack, weather, scrapers such as Apify in the later demo) are how Chat/Cowork reach proprietary or live systems—usually MCP under the product name “connectors.”

Who decides to call a tool?

A precise classroom answer: the **model produces a plan** (todos). The **client executes** steps. Some steps are tool/HTTP with no extra LLM; some steps need another LLM pass (extract “rainy” from a fat JSON, then generate an image background). Anthropic does not fully document whether a given call is “model-native tools” or “client tools”; Vijay’s working assumption for Claude was client-side execution after the plan. Exam language: know that *planning is model; execution is the product*.

Usage you can see as an end user

Settings → Usage showed a percentage of a rolling window (classroom: on the order of a five-hour reset plus a weekly cap; exact numbers are plan- and admin-specific). Team accounts may hide per-user token counts unless you are admin. Enterprise UST licences were described as org-aggregated; token spend will become a KPI. If you exhaust quota mid-day, extra tokens are an org procurement issue, not a “refill button” on Chat.

STUDY MOVE

Open Chat once this week: pick a model, ask the cutoff date, toggle web search, attach a harmless public PDF, then start a fresh thread. You are practising product literacy the Foundation exam assumes —not coding yet.

Official documentation

- **Claude Docs home**

<https://docs.claude.com/en/home>

Product and API docs; use when Chat UI labels differ from class.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

How workspace / project grouping is described in current docs.

- **Vision**

<https://docs.claude.com/en/docs/build-with-claude/vision>

Images in Chat and in the API.

- **PDF support**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Document context beyond a pasted paragraph.

- **Extended thinking**

<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>

The ‘thinking’ toggle Vijay tied to more expensive models.

- **Help: where is the API documentation?**

<https://support.claude.com/en/articles/8114490-where-can-i-find-your-api-documentation>

If you only have the consumer Chat UI and need the API path.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.